

DSA Mastery Notes

Every Pattern You Need to Crack Product-Based Company Interviews

Python Implementations · Pseudocode · Step-by-Step Visual Traces

Beginner → Advanced

14+ Core Patterns

Python 3

Table of Contents

- P.** Python Prerequisites (Variables, Lists, Arrays, Strings, Classes, Built-ins)
- 0.** How to Use These Notes + Big-O Foundations
- 1.** Two Pointers
- 2.** Sliding Window
- 3.** Prefix Sum
- 4.** Frequency Counting & Hashing
- 5.** Matrix Traversal
- 6.** Stack & Monotonic Stack
- 7.** Linked List Patterns
- 8.** Binary Trees & Traversals (Pre/In/Post/Level)
- 9.** Binary Search Trees
- 10.** Heaps & Top-K Elements
- 11.** Graphs: BFS & DFS
- 12.** Backtracking
- 13.** Greedy Algorithms
- 14.** Dynamic Programming (1D, 2D, Knapsack, LCS, etc.)
- 15.** Overlapping Intervals
- 16.** Binary Search on Answer
- 17.** Trie (Prefix Tree)
- 18.** Bit Manipulation
- 19.** Union-Find (Disjoint Set)
- 20.** Final Interview Playbook & Practice Roadmap

P. Python Prerequisites

Before diving into patterns, make sure these Python fundamentals feel automatic. Interviewers expect you to write clean Python without pausing to recall syntax.

P.1 Variables & Basic Types

```
x = 10                # int
y = 3.14              # float
name = "Claude"       # str
is_valid = True        # bool
nothing = None         # NoneType

print(type(x))        # <class 'int'>
a, b = 5, 10          # multiple assignment
a, b = b, a            # swap without a temp variable!
```

P.2 Lists — the Python "array" you'll use 90% of the time

A Python `list` is a dynamic, resizable array. It can hold mixed types, though in DSA we usually keep one type per list.

```
nums = [5, 2, 9, 1]

# Indexing & slicing
nums[0]      # 5      (first element)
nums[-1]     # 1      (last element)
nums[1:3]    # [2, 9] (index 1 up to, not including, 3)
nums[:2]     # [5, 2] (from start)
nums[2:]     # [9, 1] (to end)
nums[::-1]   # [1, 9, 2, 5] (reversed copy)

len(nums)    # 4
```

P.3 Essential List Built-in Methods

Method	What it does	Time Complexity
<code>nums.append(x)</code>	Add x to the end	O(1) amortized
<code>nums.pop()</code>	Remove & return last element	O(1)
<code>nums.pop(i)</code>	Remove & return element at index i	O(n)
<code>nums.insert(i, x)</code>	Insert x at index i	O(n)
<code>nums.remove(x)</code>	Remove first occurrence of value x	O(n)
<code>nums.sort()</code>	Sort in place, ascending	O(n log n)
<code>nums.sort(reverse=True)</code>	Sort in place, descending	O(n log n)
<code>sorted(nums)</code>	Return a NEW sorted list (original untouched)	O(n log n)
<code>nums.reverse()</code>	Reverse in place	O(n)
<code>nums.index(x)</code>	Find first index of value x	O(n)
<code>x in nums</code>	Membership check	O(n)
<code>nums.count(x)</code>	Count occurrences of x	O(n)
<code>nums.extend(other)</code>	Append all items from another list	O(len(other))
<code>nums.copy()</code> / <code>nums[:]</code>	Shallow copy	O(n)
<code>[0]*n</code>	Create list of n zeros	O(n)
<code>[[0]*cols for _ in range(rows)]</code>	Create a 2D grid (never use <code>[[0]*cols]*rows</code> — that shares references between rows!)	O(rows*cols)

Common trap: `grid = [[0]*3]*3` creates 3 references to the *same* inner list. Changing `grid[0][0]` changes every row! Always use a list comprehension: `[[0]*3 for _ in range(3)]`.

P.4 The array Module vs Python list

Python's built-in `list` is what most interview code uses — it's flexible and simple. The `array` module (and `numpy`) exist for storing large amounts of a single numeric type more memory-efficiently, but you rarely need them for coding interviews.

```
import array
arr = array.array('i', [1, 2, 3]) # 'i' = signed int, memory-efficient, fixed type
# In interviews, just use a normal list unless the interviewer specifies otherwise.
```

P.5 Strings

Strings are immutable — every "modification" actually creates a new string.

```
s = "hello world"
s.upper()          # "HELLO WORLD"
s.lower()          # "hello world"
s.split()          # ["hello", "world"] (splits on whitespace by default)
s.split(",")       # split on a specific character
"-".join(["a","b"]) # "a-b"
s.strip()          # remove leading/trailing whitespace
s.replace("l", "L") # "heLlo worLd"
s[::-1]            # reverse a string
s.find("wor")      # 6 (index, or -1 if not found)
ord('a')           # 97 (char -> ASCII code)
chr(97)            # 'a' (ASCII code -> char)
f"{s} has {len(s)} chars" # f-strings for formatting
```

Interview tip: Since strings are immutable, repeatedly doing `s += ch` in a loop is $O(n)$ per operation ($O(n^2)$ total). Instead, collect characters in a list and `''.join(list)` at the end — $O(n)$ total.

P.6 Tuples, Sets, and Dictionaries

```
# Tuple: ordered, IMMUTABLE -- great as dict keys or fixed records
point = (3, 4)
x, y = point # unpacking

# Set: unordered, unique elements, O(1) average membership check
s = {1, 2, 3}
s.add(4)
s.discard(2) # remove if present, no error if missing
s2 = {2, 3, 5}
s & s2       # intersection {3}
s | s2       # union {1,3,4,5, ...}
s - s2       # difference

# Dictionary: key -> value, O(1) average lookup/insert
d = {"a": 1, "b": 2}
d["c"] = 3 # insert/update
d.get("z", 0) # returns 0 if "z" not found (no KeyError)
d.keys(), d.values(), d.items() # views for iteration
for key, value in d.items():
    print(key, value)
"a" in d # membership check on KEYS, O(1)
```

P.7 Loops, Comprehensions & Common Built-ins

```
for i in range(5): # 0,1,2,3,4
for i in range(2, 10, 2): # 2,4,6,8 (start, stop, step)
for i, val in enumerate(nums): # index + value together
for a, b in zip(list1, list2): # pair up two lists element-wise

squares = [x*x for x in range(5)] # list comprehension: [0,1,4,9,16]
evens = [x for x in nums if x % 2 == 0] # with a filter condition

max(nums), min(nums), sum(nums) # aggregate built-ins
```

```
sorted(nums, key=lambda x: -x)      # custom sort key (here: descending)
map(str, nums)                      # apply a function to every element (returns iterator)
filter(lambda x: x > 0, nums)       # keep elements matching a condition (returns iterator)
any(x > 10 for x in nums)           # True if ANY element satisfies condition
all(x > 0 for x in nums)            # True if ALL elements satisfy condition
```

P.8 Functions

```
def add(a, b=0):                    # b has a default value
    return a + b

def variadic(*args, **kwargs):
    print(args)                    # tuple of positional args
    print(kwargs)                  # dict of keyword args

square = lambda x: x * x           # anonymous one-line function
```

P.9 Classes & Object-Oriented Basics

You will define small classes constantly in DSA (e.g. `ListNode`, `TreeNode`). Here's everything you need:

```
class Animal:
    species_count = 0              # CLASS variable, shared by all instances

    def __init__(self, name, sound):
        self.name = name          # INSTANCE variable, unique per object
        self.sound = sound
        Animal.species_count += 1

    def make_sound(self):          # instance method (needs 'self')
        return f"{self.name} says {self.sound}"

    def __repr__(self):            # controls how the object prints (great for debugging)
        return f"Animal({self.name})"

    def __eq__(self, other):       # controls how == compares two objects
        return self.name == other.name

    def __lt__(self, other):       # controls how < compares -- needed for sorting/heapq!
        return self.name < other.name

class Dog(Animal):                # INHERITANCE: Dog gets everything Animal has
    def __init__(self, name):
        super().__init__(name, "Woof") # call the parent constructor

    def fetch(self):              # method unique to Dog
        return f"{self.name} fetches the ball!"

d = Dog("Rex")
print(d.make_sound())            # "Rex says Woof" (inherited method)
print(d.fetch())                 # "Rex fetches the ball!"
```

Why this matters for DSA: Every linked-list/tree problem starts with a tiny class like `ListNode` or `TreeNode` (shown in the sections ahead). Custom objects also need `__lt__` defined if you want to push them into a `heapq`, and `__eq__`/`__hash__` if you want to put them in a `set` or use them as dict keys.

P.10 Mutable vs Immutable & Pass-by-Reference Gotchas

Mutable (can change in place)	Immutable (cannot change in place)
list, dict, set, custom objects	int, float, str, tuple, bool, frozenset

```
def modify(lst):
    lst.append(4)                  # this DOES affect the caller's list (mutable, passed by reference)

def reassign(lst):
    lst = [9, 9, 9]               # this does NOT affect the caller (just rebinds the local name)
```

```
nums = [1, 2, 3]
modify(nums)      # nums is now [1, 2, 3, 4]
reassign(nums)    # nums is STILL [1, 2, 3, 4] -- reassignment inside a function is local
```

P.11 Recursion Basics (used constantly in Trees, Backtracking, DP)

```
def factorial(n):
    if n <= 1:                # BASE CASE -- always define this first, or you get infinite recursion
        return 1
    return n * factorial(n - 1)  # RECURSIVE CASE -- problem shrinks toward the base case
```

Recursion checklist: (1) What's the base case? (2) What does this function promise to return for a smaller input? (3) How do I combine the smaller answer into the current answer? If you can answer these three questions, you can write any recursive function.

0. How to Use These Notes + Big-O Foundations

0.1 How to study this document

Every pattern below follows the same structure so you can build a repeatable habit:

1. **What it is** — the core idea in plain English.
2. **When to use it** — the "signal words" in a problem that hint you should use this pattern.
3. **Pseudocode** — language-agnostic steps.
4. **Python implementation** — a clean, interview-ready template.
5. **Step-by-step visual trace** — walking through a real example one step at a time.
6. **Complexity** — time and space.
7. **Practice problems** — easy → hard, the kind product companies (Google, Amazon, Microsoft, Flipkart, etc.) actually ask.

Golden rule: Don't memorize code. Memorize the *decision process*: "I see two sorted arrays and need pairs → Two Pointers." "I see a subarray/substring with a condition → Sliding Window." Patterns are shortcuts your brain uses to avoid re-inventing an algorithm every time.

0.2 What is Big-O Notation?

Big-O describes how the **running time** or **memory** of an algorithm grows as the input size (n) grows. It ignores constants and lower-order terms because we care about behavior at large scale.

	$n = 1$	$n = 10$	$n = 1000$	
$O(1)$	1	1	1	(constant)
$O(\log n)$	1	3	10	(grows very slowly)
$O(n)$	1	10	1000	(linear)
$O(n \log n)$	1	33	~9966	(linearithmic)
$O(n^2)$	1	100	1,000,000	(quadratic)
$O(2^n)$	2	1024	1.07×10^{301}	(exponential - avoid!)

Complexity	Name	Typical Pattern
$O(1)$	Constant	Array index access, hash map lookup
$O(\log n)$	Logarithmic	Binary search, balanced BST operations
$O(n)$	Linear	Single loop over input
$O(n \log n)$	Linearithmic	Efficient sorting (merge/heap sort)
$O(n^2)$	Quadratic	Nested loops, brute-force pairs
$O(2^n)$	Exponential	Brute-force subsets, unoptimized recursion
$O(n!)$	Factorial	Brute-force permutations

0.3 Space Complexity

Same idea, but for memory: how much **extra** space (not counting the input itself) does your algorithm use? A recursive solution has $O(\text{depth})$ space from the call stack; an iterative solution with a hash map has $O(n)$ space for the map.

0.4 Core Data Structures Cheat Sheet

Structure	Access	Search	Insert	Delete	Use When
Array / List	$O(1)$	$O(n)$	$O(n)$	$O(n)$	Fixed-order data, random access
Dynamic Array (Python list)	$O(1)$	$O(n)$	$O(1)$ amortized end	$O(n)$	General purpose
Linked List	$O(n)$	$O(n)$	$O(1)$ at known node	$O(1)$ at known node	Frequent insert/delete at ends

Hash Map / Set	-	O(1) avg	O(1) avg	O(1) avg	Fast lookups, counting, dedup
Stack	O(n)	O(n)	O(1) top	O(1) top	LIFO, matching brackets, undo
Queue / Deque	O(n)	O(n)	O(1) ends	O(1) ends	FIFO, BFS, sliding window
Binary Heap	O(1) top	O(n)	O(log n)	O(log n)	Priority, Top-K, scheduling
Binary Search Tree	O(log n) avg	O(log n) avg	O(log n) avg	O(log n) avg	Sorted dynamic data
Trie	O(L)	O(L)	O(L)	O(L)	Prefix search, autocomplete
Graph (adj list)	-	O(V+E)	O(1)	O(E)	Relationships/networks

L = length of the string/key, V = vertices, E = edges.

Interview tip: Always state the brute-force approach first (even $O(n^2)$), then say "we can optimize using X pattern" and derive the better solution out loud. Interviewers grade your *thinking process*, not just the final code.

1. Two Pointers

1.1 What it is

Use two index variables (pointers) that move through a data structure — usually a sorted array or string — to avoid a nested loop. Instead of checking every pair ($O(n^2)$), the pointers move intelligently based on a comparison, giving $O(n)$.

Signal words: "sorted array", "pair that sums to X", "palindrome", "remove duplicates in-place", "container with most water", "merge two sorted lists".

1.2 Two Common Shapes

Shape	Description	Example
Opposite ends	left starts at 0, right starts at n-1, they move toward each other	Two Sum (sorted), palindrome check, container with most water
Same direction (fast/slow)	both start at 0, move at different speeds or one waits	Remove duplicates, cycle detection, partitioning

1.3 Pseudocode (opposite ends)

```
left = 0
right = length(array) - 1
while left < right:
    if condition_met(array[left], array[right]):
        record answer
        move both pointers
    elif array[left] + array[right] < target:
        left += 1      # need a bigger sum
    else:
        right -= 1     # need a smaller sum
```

1.4 Python Implementation — Two Sum on Sorted Array

```
def two_sum_sorted(nums, target):
    """
    Given a SORTED array, find indices of two numbers that add to target.
    Time: O(n) -- single pass, each pointer moves at most n times
    Space: O(1) -- no extra data structure
    """
    left, right = 0, len(nums) - 1

    while left < right:
        current_sum = nums[left] + nums[right]
        if current_sum == target:
            return [left, right]
        elif current_sum < target:
            left += 1 # sum too small -> increase it
        else:
            right -= 1 # sum too big -> decrease it

    return [-1, -1] # no pair found

# Example
print(two_sum_sorted([2, 7, 11, 15, 20], 26)) # [1, 3] (7 + 15 + wait check below)
```

1.5 Step-by-Step Visual Trace

Array = [2, 7, 11, 15], target = 18

```
Step 1:  [2, 7, 11, 15]      left=0(2)  right=3(15)  sum=17 < 18  -> left++
                L                R
```

```
Step 2:  [2, 7, 11, 15]      left=1(7) right=3(15)    sum=22 > 18  -> right--
                L           R
```

```
Step 3:  [2, 7, 11, 15]      left=1(7) right=2(11)    sum=18 == 18 -> FOUND! [1,2]
                L           R
```

1.6 Python Implementation — Valid Palindrome (opposite ends)

```
def is_palindrome(s):
    s = ''.join(ch.lower() for ch in s if ch.isalnum())
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True
```

1.7 Python Implementation — Remove Duplicates (fast/slow, same direction)

```
def remove_duplicates(nums):
    """
    nums is sorted. Overwrite duplicates in-place.
    slow = position of the last unique element found
    fast = scans ahead looking for the next unique element
    """
    if not nums:
        return 0
    slow = 0
    for fast in range(1, len(nums)):
        if nums[fast] != nums[slow]:
            slow += 1
            nums[slow] = nums[fast]
    return slow + 1  # new length
```

```
nums = [1, 1, 2, 2, 3]
slow=0 fast=1: nums[1]==nums[0] -> skip
slow=0 fast=2: nums[2]!=nums[0] -> slow=1, nums[1]=2 => [1,2,2,2,3]
slow=1 fast=3: nums[3]==nums[1] -> skip
slow=1 fast=4: nums[4]!=nums[1] -> slow=2, nums[2]=3 => [1,2,3,2,3]
Result length = 3 -> first 3 elements [1,2,3] are the unique sorted values
```

1.8 Complexity

Time: $O(n)$ — each pointer traverses the array at most once.

Space: $O(1)$ — only a couple of index variables.

1.9 Practice Problems (Easy → Hard)

Easy: Valid Palindrome · Two Sum II (sorted input) · Remove Duplicates from Sorted Array · Move Zeroes

Medium: 3Sum · Container With Most Water · Sort Colors (Dutch National Flag) · Trapping Rain Water (two-pointer variant)

Hard: 4Sum · Minimum Window covering all characters (hybrid with sliding window) · Merge two sorted arrays in-place from the end

2. Sliding Window

2.1 What it is

A "window" is a contiguous range `[left, right]` inside an array/string. Instead of recomputing a sum/count from scratch for every possible window ($O(n^2)$), we **slide** the window: expand the right edge to include new elements and shrink the left edge to remove old ones, updating a running result incrementally.

Signal words: "contiguous subarray/substring", "longest/shortest/maximum/minimum window", "at most K distinct", "sum equals/at least/at most K".

2.2 Two Flavors

Type	When	Example
Fixed-size window	Window size K is given	Max sum subarray of size K
Variable-size window	Grow/shrink based on a condition	Longest substring without repeating characters

2.3 Pseudocode (variable window)

```
left = 0
window_state = {}      # e.g., counts, sum
for right in range(n):
    add array[right] to window_state

    while window_is_invalid(window_state):
        remove array[left] from window_state
        left += 1

    update_answer(window_state, left, right)
```

2.4 Python Implementation — Longest Substring Without Repeating Characters

```
def length_of_longest_substring(s):
    """
    Time: O(n) -- each character is added once and removed at most once
    Space: O(min(n, charset)) -- the set holds unique characters in window
    """
    seen = set()
    left = 0
    best = 0

    for right in range(len(s)):
        # Shrink from the left until s[right] is no longer a duplicate
        while s[right] in seen:
            seen.remove(s[left])
            left += 1

        seen.add(s[right])
        best = max(best, right - left + 1)

    return best
```

2.5 Step-by-Step Visual Trace

S = "abcabcbb"

right=0 'a': window="a"	seen={a}	best=1
right=1 'b': window="ab"	seen={a,b}	best=2
right=2 'c': window="abc"	seen={a,b,c}	best=3
right=3 'a': DUPLICATE 'a' -> remove s[left]='a', left=1		
window="bca"	seen={b,c,a}	best=3
right=4 'b': DUPLICATE 'b' -> remove s[left]='b', left=2		

```

        window="cab"          seen={c,a,b}          best=3
right=5 'c': DUPLICATE 'c' -> remove s[left]='c', left=3
        window="abc"          seen={a,b,c}          best=3
right=6 'b': DUPLICATE 'b' -> remove s[left]='a', left=4
        remove s[left]='b', left=5
        window="cb"           seen={c,b}           best=3
right=7 'b': DUPLICATE 'b' -> remove s[left]='c', left=7
        window="bb"->fix: remove 'b', left=7
        window="b"            seen={b}             best=3
        Answer: 3   ("abc")

```

2.6 Python Implementation — Fixed Size Window (Max Sum of Size K)

```

def max_sum_subarray(nums, k):
    window_sum = sum(nums[:k])
    best = window_sum
    for right in range(k, len(nums)):
        window_sum += nums[right] - nums[right - k]    # add new, drop oldest
        best = max(best, window_sum)
    return best

```

2.7 Python Implementation — Minimum Window Substring (Hard, classic)

```

from collections import Counter

def min_window(s, t):
    if not t or not s:
        return ""

    need = Counter(t)
    missing = len(t)          # total characters still needed
    left = 0
    best_left, best_len = 0, float('inf')

    for right, ch in enumerate(s):
        if need[ch] > 0:
            missing -= 1
            need[ch] -= 1

        while missing == 0:    # window is valid, try to shrink
            if right - left + 1 < best_len:
                best_left, best_len = left, right - left + 1
            need[s[left]] += 1
            if need[s[left]] > 0:
                missing += 1
            left += 1

    return "" if best_len == float('inf') else s[best_left: best_left + best_len]

```

2.8 Complexity

Time: $O(n)$ — left and right pointers each move forward at most n times, total $O(2n) = O(n)$.

Space: $O(k)$ where k is the size of the character set / map being tracked.

2.9 Practice Problems (Easy → Hard)

Easy: Maximum Sum Subarray of Size K · Average of Subarrays of Size K

Medium: Longest Substring Without Repeating Characters · Longest Repeating Character Replacement · Permutation in String · Fruit Into Baskets (at most 2 distinct)

Hard: Minimum Window Substring · Sliding Window Maximum (uses monotonic deque) · Substring with Concatenation of All Words

3. Prefix Sum

3.1 What it is

A prefix sum array stores the running total up to each index. Once built, the sum of any subarray `[i, j]` can be found in $O(1)$ instead of re-adding elements every time.

Signal words: "range sum query", "subarray sum equals K", "multiple queries about sums of ranges".

3.2 The Core Formula

```
prefix[0] = 0
prefix[i] = prefix[i-1] + nums[i-1]

sum(nums[i..j]) = prefix[j+1] - prefix[i]
```

3.3 Python Implementation — Range Sum Query

```
class PrefixSum:
    def __init__(self, nums):
        self.prefix = [0] * (len(nums) + 1)
        for i, num in enumerate(nums):
            self.prefix[i + 1] = self.prefix[i] + num

    def range_sum(self, i, j):
        """Sum of nums[i..j] inclusive, O(1) per query."""
        return self.prefix[j + 1] - self.prefix[i]

# Example
nums = [2, 4, 5, 1, 3]
ps = PrefixSum(nums)
print(ps.range_sum(1, 3))  # 4 + 5 + 1 = 10
```

3.4 Step-by-Step Visual Trace

```
nums:    [ 2,  4,  5,  1,  3]
index:    0   1   2   3   4

prefix: [0, 2, 6, 11, 12, 15]
index:  0  1  2  3  4  5
```

```
range_sum(1,3) = prefix[4] - prefix[1] = 12 - 2 = 10
                (matches 4+5+1 = 10)  correct!
```

3.5 Python Implementation — Subarray Sum Equals K (uses prefix sum + hashmap)

```
from collections import defaultdict

def subarray_sum(nums, k):
    """
    Count how many contiguous subarrays sum to k.
    Idea: if prefix[j] - prefix[i] = k, then prefix[i] = prefix[j] - k.
    Store how many times each prefix sum has occurred.
    Time: O(n)
    Space: O(n)
    """
    count_map = defaultdict(int)
    count_map[0] = 1  # empty prefix sums to 0
    running_sum = 0
    result = 0
```

```

for num in nums:
    running_sum += num
    result += count_map[running_sum - k]    # how many earlier prefixes make this valid
    count_map[running_sum] += 1

return result

print(subarray_sum([1, 1, 1], 2))    # 2    -> [1,1] appears twice

```

3.6 2D Prefix Sum (Matrix Range Sum)

```

def build_2d_prefix(matrix):
    rows, cols = len(matrix), len(matrix[0])
    prefix = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(rows):
        for c in range(cols):
            prefix[r+1][c+1] = (matrix[r][c]
                                + prefix[r][c+1]
                                + prefix[r+1][c]
                                - prefix[r][c])

    return prefix

def region_sum(prefix, r1, c1, r2, c2):
    # sum of matrix[r1..r2][c1..c2], inclusive, 0-indexed
    return (prefix[r2+1][c2+1] - prefix[r1][c2+1]
            - prefix[r2+1][c1] + prefix[r1][c1])

```

3.7 Complexity

Build: $O(n)$ time, $O(n)$ space.

Each query: $O(1)$ time after the prefix array is built — huge win when there are many queries.

3.8 Practice Problems (Easy → Hard)

Easy: Running Sum of 1D Array · Range Sum Query - Immutable

Medium: Subarray Sum Equals K · Product of Array Except Self · Contiguous Array (equal 0s and 1s)

Hard: Range Sum Query 2D - Immutable · Maximum Size Subarray Sum Equals K · Count of Range Sum

4. Frequency Counting & Hashing

4.1 What it is

Use a hash map (Python `dict`) or hash set (`set`) to record how many times something appears, or whether it has been seen, in $O(1)$ average time per lookup. This trades space for massive time savings versus nested loops.

Signal words: "anagram", "duplicate", "frequency", "count occurrences", "group by", "have I seen this before".

4.2 Python's Counter — your best friend

```
from collections import Counter

def is_anagram(s, t):
    """
    Time: O(n)
    Space: O(1) -- at most 26 lowercase letters
    """
    return Counter(s) == Counter(t)

print(is_anagram("listen", "silent")) # True
```

4.3 Python Implementation — Two Sum (unsorted array, using hashmap)

```
def two_sum(nums, target):
    """
    Time: O(n) -- single pass
    Space: O(n) -- hashmap of seen values
    """
    seen = {} # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

4.4 Step-by-Step Visual Trace

nums = `[2, 7, 11, 15]`, target = `9`

i=0 num=2	complement=7	seen={}	-> not found, add seen={2:0}
i=1 num=7	complement=2	seen={2:0}	-> FOUND 2 at index 0! return [0, 1]

4.5 Python Implementation — Group Anagrams

```
from collections import defaultdict

def group_anagrams(strs):
    """
    Key idea: two words are anagrams if their SORTED letters match.
    Use the sorted string as a hashmap key to bucket words together.
    Time: O(n * k log k) -- n words, k = avg word length (sorting cost)
    Space: O(n * k)
    """
    groups = defaultdict(list)
    for word in strs:
        key = ''.join(sorted(word))
        groups[key].append(word)
    return list(groups.values())
```

4.6 Python Implementation — First Non-Repeating Character

```

from collections import Counter

def first_unique_char(s):
    freq = Counter(s)
    for i, ch in enumerate(s):
        if freq[ch] == 1:
            return i
    return -1

```

4.7 Hash Map vs Hash Set Cheat Sheet

Use a Set when	Use a Dict when
You only need to check "have I seen this?"	You need to track a count or map a key to extra info
Detecting duplicates	Frequency counting, memoization, grouping
Fast membership tests for graph visited nodes	Two Sum style "value -> index" lookups

4.8 Complexity

Time: $O(1)$ average per insert/lookup, $O(n)$ to build over the whole input.

Space: $O(n)$ in the worst case for the map/set.

Warning: Worst-case hash collisions can degrade to $O(n)$ per operation, but this is rare in practice and not a concern in interviews.

4.9 Practice Problems (Easy → Hard)

Easy: Two Sum · Valid Anagram · Contains Duplicate · First Unique Character in a String

Medium: Group Anagrams · Top K Frequent Elements · Longest Consecutive Sequence · Subarray Sum Equals K

Hard: LRU Cache (hashmap + doubly linked list) · Substring with Concatenation of All Words · Design Twitter

5. Matrix Traversal

5.1 What it is

A 2D grid problem, treated as a graph where each cell is a node connected to its (usually 4) neighbors. Traversal patterns include row/column scanning, spiral order, and flood-fill (BFS/DFS on a grid).

Signal words: "grid", "matrix", "islands", "spiral", "rotate image", "flood fill", "shortest path in a grid".

5.2 The 4-Direction Template

```
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)] # up, down, left, right

def in_bounds(r, c, rows, cols):
    return 0 <= r < rows and 0 <= c < cols
```

5.3 Python Implementation — Number of Islands (DFS flood fill)

```
def num_islands(grid):
    """
    grid[r][c] == '1' means land, '0' means water.
    Every time we find unvisited land, it's a new island;
    sink the entire connected island with DFS so we don't count it again.
    Time: O(rows * cols)
    Space: O(rows * cols) worst case recursion stack
    """
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])

    def dfs(r, c):
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != '1':
            return
        grid[r][c] = '0' # mark visited by sinking it
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    islands = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                islands += 1
                dfs(r, c)
    return islands
```

5.4 Step-by-Step Visual Trace

```
Grid:          Start scanning left-to-right, top-to-bottom.
1 1 0 0        Find land at (0,0) -> island #1 -> DFS sinks
                the whole connected block:
                1 1 0 0      0 0 0 0
                  0 0 1 0      1 1 0 0      0 0 0 0
0 0 0 1        1 1 0 0 --> 0 0 0 0 (all connected 1s -> 0)
                        0 0 1 0      0 0 1 0
                        0 0 0 1      0 0 0 1

Continue scanning: find land at (2,2) -> island #2 -> sink it.
Continue scanning: find land at (3,3) -> island #3 -> sink it.

Total islands = 3
```

5.5 Python Implementation — Rotate Image 90 Degrees (in-place)

```
def rotate(matrix):
    """
    Transpose the matrix, then reverse each row.
    Time: O(n^2)
    Space: O(1) -- in place
    """
    n = len(matrix)
    # Step 1: transpose (swap matrix[i][j] with matrix[j][i])
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

    # Step 2: reverse every row
    for row in matrix:
        row.reverse()
```

5.6 Python Implementation — Spiral Matrix Traversal

```
def spiral_order(matrix):
    result = []
    if not matrix:
        return result

    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1

    while top <= bottom and left <= right:
        for c in range(left, right + 1):          # go right along top row
            result.append(matrix[top][c])
            top += 1

        for r in range(top, bottom + 1):          # go down along right col
            result.append(matrix[r][right])
            right -= 1

        if top <= bottom:
            for c in range(right, left - 1, -1): # go left along bottom row
                result.append(matrix[bottom][c])
                bottom -= 1

        if left <= right:
            for r in range(bottom, top - 1, -1): # go up along left col
                result.append(matrix[r][left])
                left += 1

    return result
```

1 2 3	Spiral order visits: 1 -> 2 -> 3 -> 6 -> 9 -> 8 -> 7 -> 4 -> 5
4 5 6	Shrinking boundary each lap: top++, right--, bottom--, left++
	7 8 9

5.7 Complexity

Time: $O(\text{rows} \times \text{cols})$ — every cell visited a constant number of times.

Space: $O(\text{rows} \times \text{cols})$ worst case for DFS recursion stack or BFS queue; $O(1)$ for simple scans/rotations.

5.8 Practice Problems (Easy → Hard)

Easy: Transpose Matrix · Flood Fill · Toeplitz Matrix

Medium: Number of Islands · Rotate Image · Spiral Matrix · Set Matrix Zeroes · Word Search

Hard: Max Area of Island · Shortest Path in Binary Matrix (BFS) · Dungeon Game · Making A Large Island

6. Stack & Monotonic Stack

6.1 What a stack is

A stack is Last-In-First-Out (LIFO). In Python, a normal `list` works fine as a stack: `append()` to push, `pop()` to pop the top.

Signal words: "matching brackets/parentheses", "next greater/smaller element", "undo operation", "evaluate expression", "largest rectangle in histogram".

6.2 Python Implementation — Valid Parentheses

```
def is_valid(s):
    """
    Time: O(n)
    Space: O(n) worst case (all opening brackets)
    """
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'

    for ch in s:
        if ch in '([{':
            stack.append(ch)
        else:
            # closing bracket
            if not stack or stack.pop() != pairs[ch]:
                return False

    return len(stack) == 0
```

6.3 What is a Monotonic Stack?

A monotonic stack keeps its elements always increasing (or always decreasing) from bottom to top. Whenever a new element would break that order, we pop elements off until the order is restored — and while popping, we learn something useful (like "the next greater element" for the popped item).

6.4 Python Implementation — Next Greater Element

```
def next_greater_elements(nums):
    """
    For each element, find the next element to its right that is bigger.
    If none exists, use -1.
    Time: O(n) -- each element is pushed and popped at most once
    Space: O(n) -- stack + result array
    """
    n = len(nums)
    result = [-1] * n
    stack = [] # stores INDICES, values are decreasing bottom to top

    for i in range(n):
        # while current number is bigger than the top of stack's number,
        # the current number IS the "next greater" for that index
        while stack and nums[i] > nums[stack[-1]]:
            idx = stack.pop()
            result[idx] = nums[i]
        stack.append(i)

    return result
```

6.5 Step-by-Step Visual Trace

nums = [2, 1, 2, 4, 3]

```
i=0 val=2  stack=[]      -> push 0      stack=[0]
i=1 val=1  1 < nums[0]=2  -> push 1      stack=[0,1]
i=2 val=2  2 > nums[1]=1  -> pop 1, result[1]=2
          2 == nums[0]=2 (not strictly greater) -> push 2
```

```

                                stack=[0,2]
i=3 val=4    4 > nums[2]=2    -> pop 2, result[2]=4
              4 > nums[0]=2    -> pop 0, result[0]=4
              stack empty      -> push 3          stack=[3]
i=4 val=3    3 < nums[3]=4    -> push 4          stack=[3,4]

End: indices left in stack [3,4] get result = -1 (no next greater)
Final result: [4, 2, 4, -1, -1]

```

6.6 Python Implementation — Largest Rectangle in Histogram (Hard classic)

```

def largest_rectangle_area(heights):
    """
    Monotonic increasing stack of indices.
    When we find a bar shorter than the stack top, that top bar can't
    extend any further right, so we finalize its rectangle area.
    Time: O(n)
    Space: O(n)
    """
    stack = []          # indices, heights increasing bottom->top
    max_area = 0
    heights = heights + [0] # sentinel to flush remaining stack

    for i, h in enumerate(heights):
        while stack and heights[stack[-1]] > h:
            height = heights[stack.pop()]
            width = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, height * width)
            stack.append(i)

    return max_area

```

6.7 Complexity

Time: $O(n)$ for monotonic stack problems — even though there's a nested while loop, each element is pushed once and popped at most once (amortized analysis).

Space: $O(n)$ for the stack.

6.8 Practice Problems (Easy → Hard)

Easy: Valid Parentheses · Baseball Game · Min Stack

Medium: Daily Temperatures · Next Greater Element I/II · Evaluate Reverse Polish Notation · Online Stock Span

Hard: Largest Rectangle in Histogram · Trapping Rain Water · Maximal Rectangle · Remove Duplicate Letters

7. Linked List Patterns

7.1 What it is

A chain of nodes where each node points to the next. Unlike arrays, there's no random access — you must walk from the head. Key tricks: **dummy head node** (simplifies edge cases), and **fast/slow pointers** (find middle, detect cycles).

Signal words: "reverse a linked list", "detect a cycle", "middle of the list", "merge two lists", "remove Nth node from end".

7.2 Node Definition

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

7.3 Python Implementation — Reverse a Linked List

```
def reverse_list(head):
    """
    Time: O(n)
    Space: O(1) iterative
    """
    prev = None
    curr = head
    while curr:
        next_node = curr.next  # save the next node before we overwrite
        curr.next = prev      # reverse the pointer
        prev = curr           # move prev forward
        curr = next_node      # move curr forward
    return prev              # prev is the new head
```

7.4 Step-by-Step Visual Trace

```
Original:  1 -> 2 -> 3 -> None

           prev=None curr=1
save next(2); 1.next=None; prev=1; curr=2
None <- 1      2 -> 3 -> None

           prev=1 curr=2
save next(3); 2.next=1; prev=2; curr=3
None <- 1 <- 2    3 -> None

           prev=2 curr=3
save next(None); 3.next=2; prev=3; curr=None
None <- 1 <- 2 <- 3

Loop ends (curr is None). Return prev = node 3 (new head).
Result: 3 -> 2 -> 1 -> None
```

7.5 Python Implementation — Detect Cycle (Floyd's Tortoise and Hare)

```
def has_cycle(head):
    """
    slow moves 1 step, fast moves 2 steps.
    If there's a cycle, fast will eventually "lap" slow and they meet.
    Time: O(n)
    Space: O(1)
    """
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
```

```

        fast = fast.next.next
        if slow == fast:
            return True
    return False

```

7.6 Python Implementation — Find Middle of Linked List

```

def find_middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow      # when fast reaches the end, slow is at the middle

```

7.7 Python Implementation — Merge Two Sorted Lists (uses dummy head)

```

def merge_two_lists(l1, l2):
    dummy = ListNode(-1)
    tail = dummy

    while l1 and l2:
        if l1.val <= l2.val:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next

    tail.next = l1 if l1 else l2      # attach whichever list has leftovers
    return dummy.next               # skip the dummy node itself

```

7.8 Python Implementation — Remove Nth Node From End (one pass)

```

def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy

    for _ in range(n):          # move fast n steps ahead
        fast = fast.next

    while fast.next:            # move both until fast hits the last node
        fast = fast.next
        slow = slow.next

    slow.next = slow.next.next  # skip the target node
    return dummy.next

```

7.9 Complexity

Time: $O(n)$ for almost all single-pass linked list operations.

Space: $O(1)$ iterative, $O(n)$ if solved recursively (call stack).

7.10 Practice Problems (Easy → Hard)

Easy: Reverse Linked List · Merge Two Sorted Lists · Linked List Cycle · Middle of the Linked List

Medium: Remove Nth Node From End · Add Two Numbers · Reorder List · Odd Even Linked List · Copy List with Random Pointer

Hard: Merge K Sorted Lists · Reverse Nodes in k-Group · LRU Cache

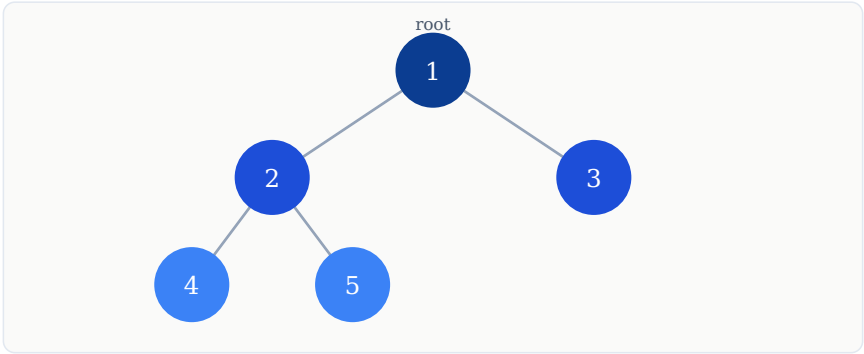
8. Binary Trees & Traversals

8.1 What it is

A tree where each node has at most two children: `left` and `right`. Almost every tree problem is a variation of a traversal (visit nodes in some order) combined with some computation at each node.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

8.2 The Four Traversal Orders



```
Preorder (Root, Left, Right):  1 -> 2 -> 4 -> 5 -> 3
Inorder  (Left, Root, Right):  4 -> 2 -> 5 -> 1 -> 3    <- sorted order for a BST!
Postorder (Left, Right, Root):  4 -> 5 -> 2 -> 3 -> 1
Level-order (BFS, by depth):    [1]  ->  [2, 3]  ->  [4, 5]
```

Order	Rule	Visit sequence on tree above	Typical use
Preorder	Root → Left → Right	1, 2, 4, 5, 3	Copy/serialize a tree
Inorder	Left → Root → Right	4, 2, 5, 1, 3	Get sorted values from a BST
Postorder	Left → Right → Root	4, 5, 2, 3, 1	Delete/free a tree, evaluate expression trees
Level-order	Breadth-first, layer by layer	1 2, 3 4, 5	Shortest path in unweighted tree, printing by level

8.3 Python Implementation — Recursive Traversals

```
def preorder(root):
    if root is None:
        return []
    return [root.val] + preorder(root.left) + preorder(root.right)

def inorder(root):
    if root is None:
        return []
    return inorder(root.left) + [root.val] + inorder(root.right)

def postorder(root):
    if root is None:
        return []
    return postorder(root.left) + postorder(root.right) + [root.val]
```

8.4 Python Implementation — Iterative Inorder (using an explicit stack)

```
def inorder_iterative(root):
    """
    Time: O(n)
    Space: O(h) where h = tree height (stack size)
```

```

"""
stack, result = [], []
curr = root

while curr or stack:
    while curr:
        # go all the way left, pushing along the way
        stack.append(curr)
        curr = curr.left

    curr = stack.pop()
    result.append(curr.val)
    curr = curr.right
    # now explore the right subtree

return result

```

8.5 Python Implementation — Level Order Traversal (BFS with a queue)

```

from collections import deque

def level_order(root):
    if not root:
        return []
    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)
        level_values = []
        for _ in range(level_size):
            node = queue.popleft()
            level_values.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level_values)

    return result

```

8.6 Step-by-Step Visual Trace (Level Order)

```

Tree:
      1
     / \
    2   3
   / \
  4   5

queue=[1]           result=[]
pop 1, add level_values=[1], push children 2,3   queue=[2,3]
level 1 done -> result=[[1]]

pop 2, level_values=[2], push 4,5
pop 3, level_values=[2,3], no children           queue=[4,5]
level 2 done -> result=[[1],[2,3]]

pop 4, level_values=[4], no children
pop 5, level_values=[4,5], no children           queue=[]
level 3 done -> result=[[1],[2,3],[4,5]]

```

8.7 Python Implementation — Maximum Depth of Binary Tree

```

def max_depth(root):
    if root is None:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))

```

8.8 Python Implementation — Check if Tree is Balanced

```
def is_balanced(root):
    def height(node):
        if node is None:
            return 0
        left_h = height(node.left)
        if left_h == -1:
            return -1                # left subtree already unbalanced
        right_h = height(node.right)
        if right_h == -1:
            return -1
        if abs(left_h - right_h) > 1:
            return -1                # this node is unbalanced
        return 1 + max(left_h, right_h)

    return height(root) != -1
```

8.9 Python Implementation — Lowest Common Ancestor (Binary Tree, not BST)

```
def lowest_common_ancestor(root, p, q):
    if root is None or root == p or root == q:
        return root

    left = lowest_common_ancestor(root.left, p, q)
    right = lowest_common_ancestor(root.right, p, q)

    if left and right:
        # p and q found in different subtrees -> this is LCA
        return root
    return left if left else right
```

8.10 Complexity

Time: $O(n)$ — every node visited once for a full traversal.

Space: $O(h)$ for recursion/explicit stack, where h = tree height ($O(\log n)$ balanced, $O(n)$ worst case skewed tree). Level order uses $O(w)$ where w = max width of the tree.

8.11 Practice Problems (Easy → Hard)

Easy: Maximum Depth of Binary Tree · Invert Binary Tree · Same Tree · Symmetric Tree

Medium: Binary Tree Level Order Traversal · Validate Binary Search Tree · Lowest Common Ancestor · Diameter of Binary Tree · Construct Tree from Preorder+Inorder

Hard: Binary Tree Maximum Path Sum · Serialize and Deserialize Binary Tree · Vertical Order Traversal

9. Binary Search Trees (BST)

9.1 What it is

A BST maintains the invariant: for every node, all values in the left subtree are smaller, and all values in the right subtree are larger. This lets us search, insert, and delete in $O(\log n)$ on a balanced tree — like binary search, but on a tree.

Signal words: "binary search tree", "kth smallest", "validate BST", "closest value", "range sum in BST".

9.2 Python Implementation — Search in a BST

```
def search_bst(root, target):
    """
    Time: O(h)  h = height (O(log n) balanced, O(n) worst case)
    Space: O(1) iterative
    """
    curr = root
    while curr:
        if curr.val == target:
            return curr
        curr = curr.left if target < curr.val else curr.right
    return None
```

9.3 Python Implementation — Insert into a BST

```
def insert_into_bst(root, val):
    if root is None:
        return TreeNode(val)
    if val < root.val:
        root.left = insert_into_bst(root.left, val)
    else:
        root.right = insert_into_bst(root.right, val)
    return root
```

9.4 Step-by-Step Visual Trace — Insert 5 into a BST

```
Start:      8                Insert 5: 5 < 8 -> go left
              / \
             3  10

Now at 3:   3                Insert 5: 5 > 3 -> go right
              / \
             1  6

Now at 6:   6                Insert 5: 5 < 6 -> go left, left is None -> place here!
              /
             5

Final tree:      8
                / \
               3  10
                / \
               1  6
                /
               5
```

9.5 Python Implementation — Validate BST (careful with bounds, not just parent comparison)

```
def is_valid_bst(root):
    def validate(node, low, high):
        if node is None:
```

```

        return True
    if not (low < node.val < high):
        return False
    return (validate(node.left, low, node.val) and
            validate(node.right, node.val, high))

return validate(root, float('-inf'), float('inf'))

```

Common mistake: Comparing a node only to its immediate parent is **wrong**. A node deep in the left subtree must be smaller than ALL its ancestors above it, not just its direct parent. Always pass down a valid (low, high) range.

9.6 Python Implementation — Kth Smallest Element in BST

```

def kth_smallest(root, k):
    """
    Inorder traversal of a BST visits nodes in SORTED order.
    So the kth value visited during inorder traversal is the answer.
    Time: O(h + k)
    Space: O(h)
    """
    stack = []
    curr = root
    count = 0

    while curr or stack:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        count += 1
        if count == k:
            return curr.val
        curr = curr.right

```

9.7 Complexity

Time: $O(\log n)$ average (balanced tree) for search/insert/delete, $O(n)$ worst case (skewed tree, essentially a linked list).

Space: $O(h)$ for recursion.

Tip: Self-balancing trees (AVL, Red-Black) guarantee $O(\log n)$ always — good to mention in interviews even if you don't implement them.

9.8 Practice Problems (Easy → Hard)

Easy: Search in a BST · Insert into a BST · Minimum Absolute Difference in BST

Medium: Validate BST · Kth Smallest Element in BST · Lowest Common Ancestor of a BST · Delete Node in a BST · Convert Sorted Array to BST

Hard: Recover Binary Search Tree · Balance a BST · Count of Smaller Numbers After Self (BST + BIT)

10. Heaps & Top-K Elements

10.1 What a heap is

A binary heap is a complete binary tree stored as an array, where every parent is smaller (min-heap) or larger (max-heap) than its children. The smallest/largest element is always at the root, accessible in $O(1)$; insert/remove are $O(\log n)$.

Signal words: "kth largest/smallest", "top K frequent", "K closest points", "merge K sorted lists", "median of a stream", "task scheduler".

10.2 Python's heapq (min-heap by default)

```
import heapq

nums = [5, 1, 8, 3, 2]
heapq.heapify(nums)          # O(n) -- rearranges list in-place into a min-heap
print(heapq.heappop(nums))   # 1 (always the smallest)
heapq.heappush(nums, 0)      # O(log n)

# For a MAX-heap, negate values on push/pop:
max_heap = []
for n in nums:
    heapq.heappush(max_heap, -n)
largest = -heapq.heappop(max_heap)
```

10.3 Python Implementation — Kth Largest Element (min-heap of size K)

```
import heapq

def find_kth_largest(nums, k):
    """
    Keep a min-heap of size k. The heap root is always the kth largest
    seen so far (because anything smaller gets kicked out).
    Time: O(n log k) -- better than sorting O(n log n) when k << n
    Space: O(k)
    """
    heap = []
    for num in nums:
        heapq.heappush(heap, num)
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0]
    # remove the smallest, keep top k largest
    # smallest of the k largest = kth largest
```

10.4 Step-by-Step Visual Trace

nums = [3, 2, 1, 5, 6, 4], k = 2 (find 2nd largest)

	push 3	heap=[3]
	push 2	heap=[2,3]
push 1	heap=[1,2,3] size=3 > k=2 -> pop smallest(1)	heap=[2,3]
push 5	heap=[2,3,5] size=3 > k=2 -> pop smallest(2)	heap=[3,5]
push 6	heap=[3,5,6] size=3 > k=2 -> pop smallest(3)	heap=[5,6]
push 4	heap=[4,5,6] size=3 > k=2 -> pop smallest(4)	heap=[5,6]

Final heap = [5, 6] -> heap[0] = 5 -> the 2nd largest element = 5
(Correct! Sorted: [1,2,3,4,5,6] -> 2nd largest is 5)

10.5 Python Implementation — Top K Frequent Elements

```
import heapq
from collections import Counter

def top_k_frequent(nums, k):
    """
```

```

Time: O(n log k)
Space: O(n)
"""
freq = Counter(nums)
# nlargest keeps a heap of size k internally
return heapq.nlargest(k, freq.keys(), key=freq.get)

```

10.6 Python Implementation — Merge K Sorted Lists (heap of pointers)

```

import heapq

def merge_k_lists(lists):
    """
    Time: O(N log k)  N = total nodes, k = number of lists
    Space: O(k) for the heap
    """
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))

    dummy = ListNode(-1)
    tail = dummy

    while heap:
        val, i, node = heapq.heappop(heap)
        tail.next = node
        tail = tail.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))

    return dummy.next

```

10.7 Python Implementation — Median of a Data Stream (two heaps)

```

import heapq

class MedianFinder:
    """
    small = max-heap (negated) holding the smaller half
    large = min-heap holding the larger half
    Keep them balanced in size so the median is always at the top(s).
    """
    def __init__(self):
        self.small = []  # max-heap (store negatives)
        self.large = []  # min-heap

    def add_num(self, num):
        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def find_median(self):
        if len(self.small) > len(self.large):
            return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2.0

```

10.8 Complexity

Time: $O(\log n)$ for push/pop, $O(n)$ to build a heap from scratch, $O(n \log k)$ for typical top-K problems.

Space: $O(n)$ for the heap, or $O(k)$ if you only maintain a bounded-size heap.

10.9 Practice Problems (Easy → Hard)

Easy: Kth Largest Element in a Stream · Last Stone Weight

Medium: Kth Largest Element in an Array · Top K Frequent Elements · K Closest Points to Origin · Task Scheduler · Reorganize String

Hard: Merge K Sorted Lists · Find Median from Data Stream · Smallest Range Covering Elements from K Lists

11. Graphs: BFS & DFS

11.1 What it is

A graph is a set of nodes (vertices) connected by edges. Trees, grids, and even state spaces (like word ladders) can be modeled as graphs. The two fundamental traversal algorithms are:

Algorithm	Data structure used	Best for
BFS (Breadth-First Search)	Queue	Shortest path in unweighted graph, level-by-level processing
DFS (Depth-First Search)	Stack / Recursion	Exploring all paths, connectivity, cycle detection, topological sort

Signal words: "shortest path" (unweighted → BFS), "connected components", "all paths", "cycle detection", "course schedule / dependency order" (topological sort).

11.2 Representing a Graph in Python

```
from collections import defaultdict

graph = defaultdict(list)
edges = [(0, 1), (0, 2), (1, 3), (2, 3)]
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u)      # remove this line for a DIRECTED graph
```

11.3 Python Implementation — BFS (Shortest Path, Unweighted)

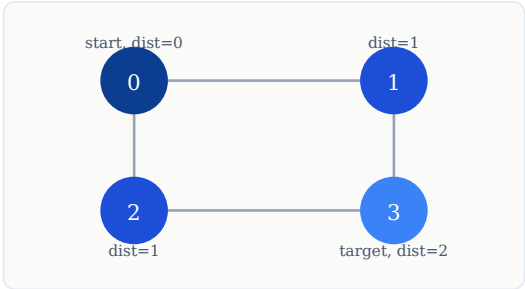
```
from collections import deque

def bfs_shortest_path(graph, start, target):
    """
    Time: O(V + E)
    Space: O(V)
    """
    visited = {start}
    queue = deque([(start, 0)])      # (node, distance)

    while queue:
        node, dist = queue.popleft()
        if node == target:
            return dist
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, dist + 1))

    return -1      # target not reachable
```

11.4 Step-by-Step Visual Trace (BFS)



Graph:

0 --- 1

| |

2 --- 3

```

Find shortest path 0 -> 3

queue=[(0,0)] visited={0}
pop (0,0): neighbors 1,2 -> add both, dist=1
queue=[(1,1),(2,1)] visited={0,1,2}

pop (1,1): neighbors 0(visited),3(new) -> add 3, dist=2
queue=[(2,1),(3,2)] visited={0,1,2,3}

pop (2,1): neighbors 0(visited),3(visited) -> nothing new
queue=[(3,2)]

pop (3,2): node == target! return dist = 2

```

11.5 Python Implementation — DFS (Recursive)

```

def dfs_recursive(graph, node, visited=None):
    if visited is None:
        visited = set()
    visited.add(node)
    print(node)                # process node

    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited)

    return visited

```

11.6 Python Implementation — DFS (Iterative, using explicit stack)

```

def dfs_iterative(graph, start):
    visited = set()
    stack = [start]

    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            for neighbor in graph[node]:
                if neighbor not in visited:
                    stack.append(neighbor)

    return visited

```

11.7 Python Implementation — Detect Cycle in Directed Graph (DFS with states)

```

def has_cycle(graph, num_nodes):
    """
    0 = unvisited, 1 = visiting (in current DFS path), 2 = fully done
    If we reach a node that is currently "visiting", we found a cycle.
    Time: O(V + E)
    """
    state = [0] * num_nodes

    def dfs(node):
        state[node] = 1
        for neighbor in graph[node]:
            if state[neighbor] == 1:
                return True                # back edge -> cycle!
            if state[neighbor] == 0 and dfs(neighbor):
                return True
        state[node] = 2
        return False

    for node in range(num_nodes):
        if state[node] == 0 and dfs(node):
            return True
    return False

```

11.8 Python Implementation — Topological Sort (Kahn's BFS algorithm)

```
from collections import deque, defaultdict

def topological_sort(num_nodes, edges):
    """
    Used for "course schedule" style dependency problems.
    Time:  $O(V + E)$ 
    """
    graph = defaultdict(list)
    indegree = [0] * num_nodes

    for u, v in edges:
        # edge u -> v means "u before v"
        graph[u].append(v)
        indegree[v] += 1

    queue = deque([n for n in range(num_nodes) if indegree[n] == 0])
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                queue.append(neighbor)

    return order if len(order) == num_nodes else [] # empty if a cycle exists
```

11.9 Complexity

Time: $O(V + E)$ for both BFS and DFS — every vertex and edge is processed once.

Space: $O(V)$ for the visited set / queue / recursion stack.

11.10 Practice Problems (Easy → Hard)

Easy: Find if Path Exists in Graph · Flood Fill · Number of Provinces

Medium: Number of Islands · Clone Graph · Course Schedule (I & II) · Rotting Oranges (multi-source BFS) · Pacific Atlantic Water Flow

Hard: Word Ladder · Alien Dictionary · Critical Connections in a Network (bridges) · Minimum Height Trees

12. Backtracking

12.1 What it is

Backtracking explores all possible choices via recursion, and **undoes** (backtracks) a choice when it leads to a dead end. Think of it as DFS over a "decision tree" of choices, with pruning to skip invalid branches early.

Signal words: "all permutations", "all subsets/combinations", "all valid arrangements", "N-Queens", "Sudoku solver", "generate parentheses", "word search".

12.2 The Universal Backtracking Template

```
def backtrack(path, choices):
    if is_solution(path):
        result.append(path.copy())    # record a valid full solution
        return

    for choice in choices:
        if not is_valid(choice, path):
            continue                # prune invalid branches early
        path.append(choice)         # MAKE the choice
        backtrack(path, next_choices) # EXPLORE further
        path.pop()                 # UNDO the choice (backtrack!)
```

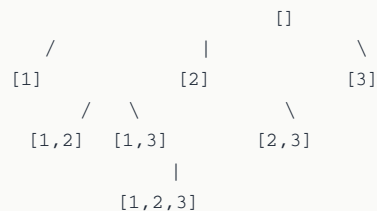
12.3 Python Implementation — Subsets (Power Set)

```
def subsets(nums):
    """
    Time: O(2^n * n) -- 2^n subsets, O(n) to copy each
    Space: O(n) recursion depth (excluding output)
    """
    result = []

    def backtrack(start, path):
        result.append(path.copy())    # every path is a valid subset
        for i in range(start, len(nums)):
            path.append(nums[i])      # choose
            backtrack(i + 1, path)    # explore
            path.pop()               # un-choose

    backtrack(0, [])
    return result
```

12.4 Step-by-Step Visual Trace — Subsets of [1,2,3]



Every node in this tree (including the root []) is a valid subset.

Order visited (DFS/preorder):

[] -> [1] -> [1,2] -> [1,2,3] -> backtrack -> [1,3] -> backtrack
-> [2] -> [2,3] -> backtrack -> [3] -> backtrack -> done

Total subsets = $2^3 = 8$

12.5 Python Implementation — Permutations

```
def permute(nums):
```

```

"""
Time: O(n! * n)
Space: O(n) recursion depth
"""

result = []

def backtrack(path, remaining):
    if not remaining:
        result.append(path.copy())
        return
    for i in range(len(remaining)):
        path.append(remaining[i])
        backtrack(path, remaining[:i] + remaining[i+1:])
        path.pop()

backtrack([], nums)
return result

```

12.6 Python Implementation — Combination Sum

```

def combination_sum(candidates, target):
    """
    Reuse of the same number is allowed (don't advance index on recursion).
    Time: O(2^target) worst case
    """
    result = []
    candidates.sort()

    def backtrack(start, path, remaining):
        if remaining == 0:
            result.append(path.copy())
            return
        for i in range(start, len(candidates)):
            if candidates[i] > remaining:
                break # prune: sorted, no point continuing
            path.append(candidates[i])
            backtrack(i, path, remaining - candidates[i]) # i, not i+1 -> reuse allowed
            path.pop()

    backtrack(0, [], target)
    return result

```

12.7 Python Implementation — N-Queens

```

def solve_n_queens(n):
    """
    Place queens row by row. Track which columns / diagonals are attacked.
    Time: O(n!) worst case, heavily pruned in practice
    """
    result = []
    cols, diag1, diag2 = set(), set(), set()
    board = [['.'] * n for _ in range(n)]

    def backtrack(row):
        if row == n:
            result.append([''.join(r) for r in board])
            return
        for col in range(n):
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue # this square is attacked, skip
            cols.add(col); diag1.add(row - col); diag2.add(row + col)
            board[row][col] = 'Q'

            backtrack(row + 1)

            cols.remove(col); diag1.remove(row - col); diag2.remove(row + col)
            board[row][col] = '.' # undo

    backtrack(0)
    return result

```

12.8 Complexity

Time: Exponential in general — $O(2^n)$ for subsets, $O(n!)$ for permutations. Pruning ("if not is_valid: continue") is what makes backtracking practical.

Space: $O(n)$ for the recursion stack and current path, plus output storage.

12.9 Practice Problems (Easy → Hard)

Easy: Binary Watch · Letter Case Permutation

Medium: Subsets · Permutations · Combination Sum · Generate Parentheses · Word Search · Palindrome Partitioning

Hard: N-Queens · Sudoku Solver · Word Search II (with Trie) · Expression Add Operators

13. Greedy Algorithms

13.1 What it is

A greedy algorithm makes the **locally optimal choice** at every step, hoping it leads to a globally optimal solution. Greedy is fast (usually $O(n \log n)$ from sorting) but only works when the problem has the **greedy-choice property** — you must be able to prove (or trust) that local optimum leads to global optimum.

Signal words: "maximum/minimum number of...", "non-overlapping", "minimum coins" (with certain coin systems), "assign/schedule to maximize", "jump game".

13.2 Python Implementation — Activity Selection / Non-overlapping Intervals

```
def erase_overlap_intervals(intervals):
    """
    Greedy idea: always keep the interval that ENDS earliest,
    because it leaves the most room for future intervals.
    Time:  $O(n \log n)$  -- dominated by the sort
    Space:  $O(1)$  extra
    """
    if not intervals:
        return 0

    intervals.sort(key=lambda x: x[1])  # sort by END time
    count_removed = 0
    last_end = intervals[0][1]

    for start, end in intervals[1:]:
        if start < last_end:  # overlaps with the last kept interval
            count_removed += 1  # remove this one (greedy: keep the earlier end)
        else:
            last_end = end  # no overlap, keep it, update last_end

    return count_removed
```

13.3 Step-by-Step Visual Trace

Intervals = `[[1,2], [2,3], [3,4], [1,3]]`

```
Sorted by end time: [1,2] [2,3] [1,3] [3,4]

last_end = 2 (from [1,2])
[2,3]: start=2 >= last_end=2 -> no overlap, keep, last_end=3
[1,3]: start=1 < last_end=3 -> OVERLAP! remove this one. count=1
[3,4]: start=3 >= last_end=3 -> no overlap, keep, last_end=4

Answer: 1 interval removed ([1,3])
```

13.4 Python Implementation — Jump Game (reach the last index?)

```
def can_jump(nums):
    """
    Greedy idea: track the farthest index reachable so far.
    If at any point our current index exceeds that farthest reach, we're stuck.
    Time:  $O(n)$ 
    """
    farthest = 0
    for i, num in enumerate(nums):
        if i > farthest:
            return False  # can't even reach this index
        farthest = max(farthest, i + num)
    return True
```

13.5 Python Implementation — Gas Station

```
def can_complete_circuit(gas, cost):
    """
    Greedy idea: if total gas >= total cost, a solution exists.
    Whenever the running tank goes negative starting from some index,
    that starting point (and everything before the failure) can't work,
    so restart from the next index.
    Time: O(n)
    """
    total_tank, curr_tank, start = 0, 0, 0

    for i in range(len(gas)):
        diff = gas[i] - cost[i]
        total_tank += diff
        curr_tank += diff
        if curr_tank < 0:
            start = i + 1      # this segment failed, try starting fresh after it
            curr_tank = 0

    return start if total_tank >= 0 else -1
```

13.6 Python Implementation — Meeting Rooms II (Minimum Rooms Needed)

```
import heapq

def min_meeting_rooms(intervals):
    """
    Greedy + heap: sort by start time; use a min-heap of end times
    to track the earliest room that will free up.
    Time: O(n log n)
    """
    if not intervals:
        return 0

    intervals.sort(key=lambda x: x[0])
    heap = []                                # stores end times of ongoing meetings

    for start, end in intervals:
        if heap and heap[0] <= start:        # earliest-ending meeting already freed up?
            heapq.heappop(heap)              # reuse that room
        heapq.heappush(heap, end)

    return len(heap)                        # rooms still occupied = rooms needed
```

13.7 Greedy vs Dynamic Programming

Greedy	Dynamic Programming
Makes one irrevocable choice per step	Considers all choices, remembers overlapping subproblems
Usually O(n log n) or O(n)	Usually O(n^2) or higher
Only works if greedy-choice property holds	Always works if optimal substructure holds
Example: Interval Scheduling	Example: 0/1 Knapsack (greedy fails here!)

13.8 Complexity

Time: Usually O(n log n) due to an initial sort, then O(n) for a single greedy pass.
Space: O(1) to O(n) depending on whether extra structures (like a heap) are needed.

13.9 Practice Problems (Easy → Hard)

- Easy:** Assign Cookies · Lemonade Change · Best Time to Buy and Sell Stock II
- Medium:** Jump Game · Gas Station · Non-overlapping Intervals · Partition Labels · Meeting Rooms II
- Hard:** Candy · Minimum Number of Arrows to Burst Balloons · Job Sequencing with Deadlines

14. Dynamic Programming (DP)

14.1 What it is

DP solves a problem by breaking it into overlapping subproblems, solving each subproblem **once**, and reusing the stored result instead of recomputing it. It requires two properties:

- **Optimal substructure:** the optimal solution to the full problem can be built from optimal solutions to subproblems.
- **Overlapping subproblems:** the same subproblem is solved multiple times in a naive recursive solution.

Signal words: "maximum/minimum ways to...", "number of distinct ways", "longest/shortest subsequence", "can you partition into...", "minimum cost to reach..."

14.2 The DP Thinking Framework (memorize this process!)

- Step 1 — Define the state: What does `dp[i]` (or `dp[i][j]`) represent in plain English?
- Step 2 — Find the recurrence: How does `dp[i]` relate to smaller states, like `dp[i-1]`?
- Step 3 — Base case(s): What is the answer for the smallest input?
- Step 4 — Order of computation: Bottom-up (iterative, fill a table) or top-down (recursive + memo cache)?
- Step 5 — Final answer: Which cell of the table holds the answer?

14.3 Two Implementation Styles

Top-Down (Memoization)	Bottom-Up (Tabulation)
Recursive, add a cache (dict or array)	Iterative, fill a table from base cases upward
Easier to write from brute force	Usually faster (no recursion overhead), easier to optimize space
Risk of stack overflow on deep recursion	No recursion depth issues

14.4 Classic Example — Fibonacci (all 3 approaches)

```
# 1) Naive recursion -- O(2^n) -- exponential, recomputes same values!
def fib_naive(n):
    if n <= 1:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)

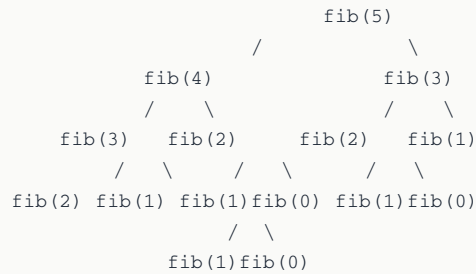
# 2) Top-down memoization -- O(n) time, O(n) space
def fib_memo(n, cache={}):
    if n <= 1:
        return n
    if n in cache:
        return cache[n]
    cache[n] = fib_memo(n - 1, cache) + fib_memo(n - 2, cache)
    return cache[n]

# 3) Bottom-up tabulation -- O(n) time, O(1) space (rolling variables)
def fib_bottom_up(n):
    if n <= 1:
        return n
    prev2, prev1 = 0, 1
    for _ in range(2, n + 1):
        prev2, prev1 = prev1, prev2 + prev1
    return prev1
```

14.5 Visual: Why Memoization Helps (Fibonacci call tree)



Naive recursion tree for fib(5) -- notice fib(3), fib(2) computed MULTIPLE times:



fib(2) is computed 3 times, fib(3) is computed 2 times -- wasted work!
Memoization stores each result the FIRST time and reuses it after that.

14.6 1D DP — House Robber

```
def rob(nums):
    """
    dp[i] = max money robbed from houses[0..i]
    Recurrence: dp[i] = max(dp[i-1], dp[i-2] + nums[i])
                  (skip house i, OR rob house i + best from i-2)
    Time: O(n)   Space: O(1) after rolling optimization
    """
    prev2, prev1 = 0, 0
    for num in nums:
        prev2, prev1 = prev1, max(prev1, prev2 + num)
    return prev1
```

```
nums = [2, 7, 9, 3, 1]
dp:    [2, 7, 11, 11, 12]
```

```
dp[0]=2
dp[1]=max(2,7)=7
dp[2]=max(7, 2+9)=11
dp[3]=max(11, 7+3)=11
dp[4]=max(11, 11+1)=12 <-- answer
```

14.7 1D DP — Longest Increasing Subsequence (LIS)

```
def length_of_lis(nums):
    """
    dp[i] = length of the longest increasing subsequence ENDING at index i
    Recurrence: dp[i] = 1 + max(dp[j]) for all j < i where nums[j] < nums[i]
    Time: O(n^2)   Space: O(n)
    (An O(n log n) version exists using binary search + patience sorting)
    """
    if not nums:
        return 0
    dp = [1] * len(nums)
    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)
```

14.8 2D DP — Longest Common Subsequence (LCS)

```
def longest_common_subsequence(text1, text2):
    """
    dp[i][j] = LCS length of text1[0..i-1] and text2[0..j-1]
    Recurrence:
        if text1[i-1] == text2[j-1]: dp[i][j] = 1 + dp[i-1][j-1]
        else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    Time: O(m*n)   Space: O(m*n) (can be optimized to O(min(m,n)))
    """
    m, n = len(text1), len(text2)
```

14.9 Step-by-Step DP Table Trace — LCS("abcde", "ace")

Reading the table: whenever row-char == col-char, we take diagonal+1.
Otherwise we take the max of the cell above or to the left.

14.10 2D DP — 0/1 Knapsack

14.11 2D DP — Edit Distance

```
        else:
            dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])

    return dp[m][n]
```

14.12 DP on Grids — Unique Paths

```
def unique_paths(m, n):
    """
    dp[r][c] = number of ways to reach cell (r,c) from (0,0), moving only right/down
    Recurrence: dp[r][c] = dp[r-1][c] + dp[r][c-1]
    Time: O(m*n)    Space: O(n) with row-compression
    """
    dp = [1] * n          # first row is always all 1s
    for r in range(1, m):
        for c in range(1, n):
            dp[c] += dp[c - 1]      # dp[c] currently holds dp[r-1][c]
    return dp[-1]
```

14.13 DP Category Cheat Sheet

Category	Classic Problem	State
1D linear	House Robber, Climbing Stairs	dp[i] depends on dp[i-1], dp[i-2]
Subsequence	LIS, LCS	dp[i] or dp[i][j] tracks best ending/matching at i,j
Knapsack (0/1 & unbounded)	Partition Equal Subset Sum, Coin Change	dp[i][capacity]
String matching/edit	Edit Distance, Wildcard Matching	dp[i][j] over two strings
Grid path	Unique Paths, Minimum Path Sum	dp[r][c]
Interval DP	Burst Balloons, Matrix Chain Multiplication	dp[i][j] over a range
State machine DP	Best Time to Buy/Sell Stock with Cooldown	dp[i][state]

14.14 Complexity

Time: Typically $O(n)$, $O(n^2)$, or $O(n \times \text{capacity})$ depending on the number of states and transitions per state.
Space: $O(\text{number of states})$; can often be reduced by only keeping the last 1-2 rows/values ("rolling array" optimization).

14.15 Practice Problems (Easy → Hard)

Easy: Climbing Stairs · House Robber · Maximum Subarray (Kadane's) · Min Cost Climbing Stairs
Medium: Coin Change · Longest Increasing Subsequence · Longest Common Subsequence · Unique Paths · Word Break · Partition Equal Subset Sum
Hard: Edit Distance · Regular Expression Matching · Burst Balloons · Best Time to Buy/Sell Stock with Cooldown · Interleaving String · Palindrome Partitioning II

15. Overlapping Intervals

15.1 What it is

Problems that give you a list of `[start, end]` ranges and ask you to merge, insert, count overlaps, or find free/busy time. The universal first step is almost always: **sort by start time** (occasionally by end time, as seen in the Greedy section).

Signal words: "merge intervals", "insert interval", "overlapping meetings", "free time", "employee schedules".

15.2 How to Tell if Two Intervals Overlap

```
Interval A: [aStart ----- aEnd]
Interval B:          [bStart ----- bEnd]

A and B overlap if:  aStart <= bEnd  AND  bStart <= aEnd
```

15.3 Python Implementation — Merge Intervals

```
def merge(intervals):
    """
    Time: O(n log n) -- dominated by sorting
    Space: O(n) for the result
    """
    if not intervals:
        return []

    intervals.sort(key=lambda x: x[0])  # sort by start time
    merged = [intervals[0]]

    for start, end in intervals[1:]:
        last_end = merged[-1][1]
        if start <= last_end:  # overlaps with the last merged interval
            merged[-1][1] = max(last_end, end)  # extend it
        else:
            merged.append([start, end])  # no overlap, start a new interval

    return merged
```

15.4 Step-by-Step Visual Trace

Intervals = `[[1,3],[2,6],[8,10],[15,18]]`

```
Sorted (already sorted by start): [1,3] [2,6] [8,10] [15,18]

merged = [[1,3]]
[2,6]: 2 <= 3 (last_end) -> overlap -> extend: merged=[[1,6]]
[8,10]: 8 <= 6? NO -> no overlap -> append: merged=[[1,6],[8,10]]
[15,18]: 15 <= 10? NO -> no overlap -> append: merged=[[1,6],[8,10],[15,18]]

Final: [[1,6],[8,10],[15,18]]
```

15.5 Python Implementation — Insert Interval

```
def insert(intervals, new_interval):
    """
    Time: O(n) -- input is already sorted, single pass
    """
    result = []
    i, n = 0, len(intervals)

    # 1) Add all intervals ending before new_interval starts
    while i < n and intervals[i][1] < new_interval[0]:
        result.append(intervals[i])
```

```

        i += 1

# 2) Merge all intervals that overlap with new_interval
while i < n and intervals[i][0] <= new_interval[1]:
    new_interval[0] = min(new_interval[0], intervals[i][0])
    new_interval[1] = max(new_interval[1], intervals[i][1])
    i += 1
result.append(new_interval)

# 3) Add all remaining intervals (start after new_interval ends)
while i < n:
    result.append(intervals[i])
    i += 1

return result

```

15.6 Python Implementation — Meeting Rooms I (can attend all meetings?)

```

def can_attend_all_meetings(intervals):
    intervals.sort(key=lambda x: x[0])
    for i in range(1, len(intervals)):
        if intervals[i][0] < intervals[i-1][1]: # overlap found
            return False
    return True

```

15.7 Complexity

Time: $O(n \log n)$ when sorting is required, $O(n)$ if the input is already sorted.

Space: $O(n)$ for the output list.

15.8 Practice Problems (Easy → Hard)

Easy: Meeting Rooms I · Summary Ranges

Medium: Merge Intervals · Insert Interval · Non-overlapping Intervals · Meeting Rooms II

Hard: Employee Free Time · Minimum Number of Arrows to Burst Balloons · Data Stream as Disjoint Intervals

16. Binary Search (Including "Binary Search on Answer")

16.1 Classic Binary Search

Repeatedly halve the search space in a sorted array by comparing the middle element to the target.

```
def binary_search(nums, target):
    """
    Time: O(log n)
    Space: O(1)
    """
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = left + (right - left) // 2    # avoids overflow (habit from other languages)
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

16.2 Step-by-Step Visual Trace

nums = [1, 3, 5, 7, 9, 11], target = 9

```

    [1, 3, 5, 7, 9, 11]    left=0 right=5 mid=2 (val=5)    5<9 -> left=3
                        L  M  R
[. . . 7, 9, 11]    left=3 right=5 mid=4 (val=9)    9==9 -> FOUND at index 4
                        L  M  R
```

16.3 The "Binary Search on Answer" Pattern

A powerful and common interview trick: when a problem asks for the **minimum/maximum value that satisfies a condition**, and that condition is "monotonic" (once true, stays true for all larger/smaller values), you can binary search over the *answer space* itself, not the array.

Signal words: "minimize the maximum...", "maximize the minimum...", "smallest capacity/speed/days such that..."

16.4 Pseudocode

```
def binary_search_on_answer(low, high, feasible):
    while low < high:
        mid = (low + high) // 2
        if feasible(mid):
            high = mid    # mid works, try to do even better
        else:
            low = mid + 1  # mid doesn't work, need a bigger value
    return low
```

16.5 Python Implementation — Koko Eating Bananas

```
import math

def min_eating_speed(piles, h):
    """
    Question: minimum eating speed k so Koko finishes all piles within h hours.
    feasible(k) = True if eating at speed k finishes in time.
    As k increases, hours needed decreases -> monotonic -> binary search works!
    Time: O(n log(max(piles)))
    """
```

```
def hours_needed(k):
    return sum(math.ceil(pile / k) for pile in piles)

low, high = 1, max(piles)
while low < high:
    mid = (low + high) // 2
    if hours_needed(mid) <= h:
        high = mid        # speed mid works, try slower (smaller)
    else:
        low = mid + 1      # too slow, need to eat faster
return low
```

16.6 Step-by-Step Visual Trace

piles = [3,6,7,11], h = 8

```

                                low=1 high=11
mid=6:  hours = ceil(3/6)+ceil(6/6)+ceil(7/6)+ceil(11/6) = 1+1+2+2 = 6 <= 8 -> feasible, high=6
                                low=1 high=6
mid=3:  hours = ceil(3/3)+ceil(6/3)+ceil(7/3)+ceil(11/3) = 1+2+3+4 = 10 > 8 -> not feasible, low=4
                                low=4 high=6
mid=5:  hours = 1+2+2+3 = 8 <= 8 -> feasible, high=5
                                low=4 high=5
mid=4:  hours = 1+2+2+3 = 8 <= 8 -> feasible, high=4
                                low=4 high=4 -> loop ends, answer = 4

```

16.7 Python Implementation — Search in Rotated Sorted Array

```
def search_rotated(nums, target):
    """
    Time: O(log n)
    Key idea: at least one half of the array (left of mid, or right of mid)
    is always properly sorted -- figure out which half, then decide where target could be.
    """
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid

        if nums[left] <= nums[mid]:        # left half is sorted
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:                               # right half is sorted
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1

    return -1
```

16.8 Complexity

Time: $O(\log n)$ for the search itself; $O(n \log(\text{range}))$ for binary-search-on-answer where each feasibility check costs $O(n)$.
Space: $O(1)$.

16.9 Practice Problems (Easy → Hard)

Easy: Binary Search · Search Insert Position · First Bad Version

Medium: Search in Rotated Sorted Array · Find Peak Element · Koko Eating Bananas · Find Minimum in Rotated Sorted Array

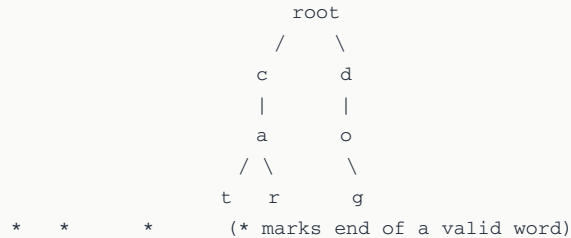
Hard: Median of Two Sorted Arrays · Split Array Largest Sum · Capacity to Ship Packages Within D Days

17. Trie (Prefix Tree)

17.1 What it is

A tree structure specialized for strings, where each path from the root spells out a prefix. Nodes are shared between words with common prefixes, making prefix search extremely fast.

Words inserted: "cat", "car", "dog"



"cat" and "car" share the path root -> c -> a

Signal words: "autocomplete", "prefix matching", "implement a dictionary of words", "word search with a list of words".

17.2 Python Implementation

```
class TrieNode:
    def __init__(self):
        self.children = {} # char -> TrieNode
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        """Time: O(L), L = length of word"""
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end_of_word = True

    def search(self, word):
        """Returns True only if the EXACT word was inserted."""
        node = self._find_node(word)
        return node is not None and node.is_end_of_word

    def starts_with(self, prefix):
        """Returns True if ANY word starts with this prefix."""
        return self._find_node(prefix) is not None

    def _find_node(self, s):
        node = self.root
        for ch in s:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node
```

17.3 Complexity

Time: O(L) for insert/search/prefix-check, where L is the length of the word/prefix — independent of how many words are stored!

Space: O(total characters across all inserted words) in the worst case (no shared prefixes).

17.4 Practice Problems (Easy → Hard)

Easy: Implement Trie (Prefix Tree) · Longest Common Prefix

Medium: Design Add and Search Words Data Structure · Replace Words · Map Sum Pairs

Hard: Word Search II · Palindrome Pairs · Stream of Characters

18. Bit Manipulation

18.1 Core Operators

Operator	Meaning	Example
&	AND — 1 only if both bits are 1	5 & 3 = 101 & 011 = 001 = 1
	OR — 1 if either bit is 1	5 3 = 101 011 = 111 = 7
^	XOR — 1 if bits differ	5 ^ 3 = 101 ^ 011 = 110 = 6
~	NOT — flips all bits	~5 = -6 (two's complement)
<<	Left shift — multiply by 2^k	5 << 1 = 10
>>	Right shift — divide by 2^k	5 >> 1 = 2

Signal words: "single number", "count set bits", "power of two", "without using +/- operators", "subsets via bitmask".

18.2 Handy Bit Tricks

```
n & (n - 1)      # removes the lowest set bit (used to count set bits, check power of 2)
n & (-n)         # isolates the lowest set bit
n ^ n == 0       # XOR-ing a number with itself cancels out to 0
n ^ 0 == n       # XOR-ing with 0 does nothing
```

18.3 Python Implementation — Single Number (every element appears twice except one)

```
def single_number(nums):
    """
    XOR-ing all numbers cancels out every pair, leaving only the unique one.
    Time: O(n)   Space: O(1)
    """
    result = 0
    for num in nums:
        result ^= num
    return result

print(single_number([4, 1, 2, 1, 2]))  # 4
```

18.4 Step-by-Step Visual Trace

```
nums = [4, 1, 2, 1, 2]

result = 0
0 ^ 4 = 4      (0100)
4 ^ 1 = 5      (0101)
5 ^ 2 = 7      (0111)
7 ^ 1 = 6      (0110)
6 ^ 2 = 4      (0100) <- final result = 4 (the unique number)
```

18.5 Python Implementation — Count Set Bits (Hamming Weight)

```
def hamming_weight(n):
    """
    Time: O(number of set bits) using the n & (n-1) trick
    """
    count = 0
    while n:
        n &= (n - 1)      # removes the lowest set bit each time
        count += 1
    return count
```

18.6 Python Implementation — Check Power of Two

```
def is_power_of_two(n):  
    # A power of two has exactly ONE set bit, e.g. 8 = 1000  
    # n & (n-1) removes that one bit, leaving 0  
    return n > 0 and (n & (n - 1)) == 0
```

18.7 Python Implementation — Generate All Subsets Using Bitmasks

```
def subsets_via_bitmask(nums):  
    """  
    Each number from 0 to 2^n - 1 represents one subset:  
    bit i is 1 means "include nums[i]".  
    Time: O(2^n * n)  
    """  
    n = len(nums)  
    result = []  
    for mask in range(1 << n):  
        # 1 << n == 2^n  
        subset = [nums[i] for i in range(n) if mask & (1 << i)]  
        result.append(subset)  
    return result
```

18.8 Complexity

Time: O(1) for basic operations, O(log n) or O(number of bits) for bit-counting loops.

Space: O(1) — bit tricks are prized for using no extra memory.

18.9 Practice Problems (Easy → Hard)

Easy: Single Number · Number of 1 Bits · Power of Two · Counting Bits

Medium: Single Number II/III · Sum of Two Integers (no +/-) · Subsets via bitmask

Hard: Maximum XOR of Two Numbers in an Array · Minimum XOR Sum of Two Arrays

19. Union-Find (Disjoint Set Union)

19.1 What it is

A structure that tracks which elements belong to the same group/set, supporting two operations very efficiently: **find** (which group is this element in?) and **union** (merge two groups). With two optimizations — **path compression** and **union by rank** — both operations run in almost $O(1)$ time.

Signal words: "connected components", "redundant connection", "accounts merge", "friend circles", "is there a cycle in an undirected graph".

19.2 Python Implementation

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))    # each node is its own parent initially
        self.rank = [0] * n

    def find(self, x):
        """Path compression: point every visited node directly to the root."""
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        """Union by rank: attach the smaller tree under the bigger tree's root."""
        root_x, root_y = self.find(x), self.find(y)
        if root_x == root_y:
            return False                # already in the same set (this would form a cycle)

        if self.rank[root_x] < self.rank[root_y]:
            root_x, root_y = root_y, root_x
        self.parent[root_y] = root_x
        if self.rank[root_x] == self.rank[root_y]:
            self.rank[root_x] += 1
        return True
```

19.3 Step-by-Step Visual Trace

```
union(0,1): parent=[1,1,2,3,4]      (0's root now points to 1)
              union(2,3): parent=[1,1,3,3,4]
union(1,2): find(1)=1, find(2)=3    parent=[1,1,3,1,4] (attach root 3 under root 1)

Now find(0): 0->1->1 (path compressed) all point directly to root 1
Sets: {0,1,2,3} and {4}
```

19.4 Python Implementation — Number of Connected Components

```
def count_components(n, edges):
    uf = UnionFind(n)
    components = n
    for u, v in edges:
        if uf.union(u, v):
            components -= 1    # successful union means we merged two separate groups
    return components
```

19.5 Complexity

Time: $O(\alpha(n))$ per operation (inverse Ackermann function) — effectively constant time in practice, with both optimizations applied.
Space: $O(n)$ for the parent and rank arrays.

19.6 Practice Problems (Easy → Hard)

Easy: Number of Provinces

Medium: Number of Connected Components · Redundant Connection · Accounts Merge · Graph Valid Tree

Hard: Number of Islands II · Swim in Rising Water · Smallest String With Swaps

20. Final Interview Playbook & Practice Roadmap

20.1 The Pattern Recognition Cheat Sheet

If you see this in the problem...	...think of this pattern
Sorted array, pair/triplet sum	Two Pointers
Contiguous subarray/substring with a condition	Sliding Window
Repeated range sum queries	Prefix Sum
Anagram, duplicate, grouping	Frequency Counting / Hashing
Grid, islands, flood fill	Matrix Traversal (BFS/DFS on grid)
Next greater/smaller element	Monotonic Stack
Reverse / cycle / middle of list	Linked List (fast-slow pointers)
Tree structure, ancestors, paths	Binary Tree Traversal (DFS/BFS)
Sorted dynamic data, kth smallest	Binary Search Tree
Kth largest/smallest, top K	Heap
Shortest path, unweighted graph	BFS
All combinations/permutations	Backtracking
"maximize/minimize" with local choice works	Greedy
Count ways, min/max cost, overlapping subproblems	Dynamic Programming
Merge/insert/count overlapping ranges	Overlapping Intervals
Minimize the maximum / feasibility check	Binary Search on Answer
Prefix-based word lookup	Trie
XOR tricks, "without arithmetic operators"	Bit Manipulation
Grouping/connectivity, cycle in undirected graph	Union-Find

20.2 A 6-Week Practice Roadmap

Week 1: Arrays & Strings basics, Two Pointers, Sliding Window, Frequency Counting/Hashing.

Week 2: Prefix Sum, Stack & Monotonic Stack, Linked Lists, Matrix Traversal.

Week 3: Binary Trees & all traversals, Binary Search Trees, Binary Search & Binary Search on Answer.

Week 4: Heaps/Top-K, Graphs (BFS/DFS), Union-Find, Trie.

Week 5: Backtracking, Greedy, Overlapping Intervals.

Week 6: Dynamic Programming (all sub-categories) — the pattern most candidates struggle with, so give it real time. Mix in mock interviews and revisit weak areas.

20.3 During the Actual Interview

- 1. Clarify first.** Ask about input size, duplicates, edge cases (empty input, negative numbers) before coding.
- 2. State brute force.** Say the $O(n^2)$ approach out loud, then say what pattern will optimize it.
- 3. Talk while you code.** Silence makes it hard for the interviewer to help or follow your reasoning.
- 4. Test with a small example.** Trace your code by hand on a tiny input before saying "I'm done."
- 5. State complexity at the end.** "This is $O(n)$ time, $O(1)$ space" shows you're thinking about efficiency.

20.4 Common Beginner Mistakes to Avoid

- Jumping to code before clarifying the problem or discussing approach.
- Forgetting edge cases: empty array, single element, all duplicates, negative numbers.
- Off-by-one errors in loop bounds — always double check `<` vs `<=`.
- Using recursion without a base case (infinite recursion / stack overflow).
- Not considering integer overflow in other languages (less of an issue in Python, but mention it if relevant).
- Memorizing solutions instead of understanding the underlying pattern — interviewers change constraints on purpose to test real understanding.

Final thought: Every hard problem is just 2-3 easy patterns combined. "Minimum Window Substring" = Sliding Window + Frequency Counting. "Word Search II" = Backtracking + Trie. Once patterns become second nature, even unfamiliar problems become a puzzle of "which patterns fit together here?" Good luck!